

TA-IND-02

Análisis de Consistencia y Replicación en la Base de Datos de TiendaTech

Estudiante: Jeremy Ruperto Gaibor Rodríguez
Equipo PFC: AGLS – TiendaTech
Asignatura: Aplicaciones Distribuidas (ISR-701)
Unidad: Unidad 3 – Bases de Datos Distribuidas
Fecha: Julio de 2026

Resumen

El presente informe analiza el modelo de consistencia y la estrategia de replicación de TiendaTech, una plataforma de comercio electrónico orientada a la venta de componentes informáticos. La implementación actual emplea una aplicación Spring Boot conectada a una única instancia PostgreSQL 17 desplegada mediante Docker Compose. Por tanto, la persistencia se encuentra centralizada y no existe replicación entre nodos. Esta configuración simplifica las operaciones transaccionales, pero convierte a PostgreSQL en un punto único de fallo. El modelo se evalúa mediante los conceptos de consistencia, CAP y PACELC. También se contrastan tres alternativas: replicación síncrona primario-respaldo, replicación asíncrona con réplicas de lectura y una estrategia híbrida. Se concluye que el diseño actual es adecuado para un prototipo académico, pero no es óptimo para una plataforma productiva que requiera alta disponibilidad. Se recomienda conservar garantías estrictas para inventario, pedidos y pagos, y utilizar réplicas asíncronas para catálogo, búsquedas y reportes.

Palabras clave: consistencia, replicación, PostgreSQL, CAP, PACELC, TiendaTech.

Índice

1. Introducción	3
2. Marco teórico	3
2.1. Modelos de consistencia	3
2.2. Teorema CAP	4
2.3. PACELC	4
2.4. Estrategias de replicación	4
2.5. Estado del arte	5
3. Análisis del modelo implementado en TiendaTech	5
3.1. Caracterización de la implementación	5
3.2. Topología de persistencia	6
3.3. Modelo de consistencia identificado	6
3.4. Estrategia de replicación actual	6
3.5. Requisitos de consistencia del dominio	7
3.6. Análisis mediante CAP	8
3.7. Análisis mediante PACELC	8
4. Evaluación de alternativas	8
4.1. Primario con respaldo síncrono	8
4.2. Primario con réplica asíncrona	8
4.3. Estrategia híbrida	8
4.4. Tabla comparativa	9
5. Discusión crítica	9
5.1. ¿Es óptimo el modelo actual?	9
5.2. Trade-offs	9
5.3. Limitaciones	10
5.4. Mitigaciones	10
5.5. Propuesta recomendada	10
6. Conclusiones	11
7. Declaración de uso de inteligencia artificial generativa	11

1. Introducción

TiendaTech es una plataforma de comercio electrónico especializada en componentes informáticos y asistencia para seleccionar productos compatibles. El sistema contempla la gestión de usuarios, catálogo, inventario, carrito de compras, pedidos, ventas y pagos. En este dominio, la persistencia debe evitar resultados contradictorios cuando varias operaciones se ejecutan simultáneamente. Los sistemas distribuidos deben equilibrar consistencia, disponibilidad, latencia y tolerancia a fallos de acuerdo con los requisitos de cada aplicación [1].

Aunque el diseño del PFC plantea una arquitectura basada en servicios, la persistencia realmente implementada utiliza una única instancia PostgreSQL. El archivo Docker Compose contiene un servicio para la aplicación y otro para la base de datos. No existe un nodo secundario al cual redirigir las operaciones si PostgreSQL deja de responder.

El uso de contenedores o infraestructura en la nube no convierte automáticamente una base de datos en distribuida. Una base distribuida requiere que los datos o las funciones de almacenamiento se encuentren coordinados entre varios nodos [1].

El objetivo de este informe es caracterizar el modelo de consistencia realmente utilizado en TiendaTech, analizar sus propiedades mediante CAP y PACELC y evaluar alternativas de replicación. La pregunta orientadora es la siguiente:

¿El modelo de consistencia elegido es el óptimo para el dominio de TiendaTech?

2. Marco teórico

2.1. Modelos de consistencia

La consistencia determina las restricciones aplicadas al orden y a la visibilidad de las operaciones. No es una propiedad únicamente binaria, sino un espectro de garantías con distintos niveles de fuerza semántica. Viotti y Vukolić organizan estos modelos según las relaciones que establecen entre lecturas, escrituras y procesos [2].

La **linealizabilidad**, también relacionada con la consistencia fuerte, exige que cada operación parezca ejecutarse instantáneamente entre su inicio y su finalización. Una lectura iniciada después de una escritura confirmada debe observar el nuevo valor. Esta garantía facilita el razonamiento sobre la aplicación, pero requiere mayor coordinación cuando existen varias réplicas [2].

La **consistencia secuencial** establece que todos los procesos deben observar las operaciones en un mismo orden global, compatible con el orden de cada proceso. A diferencia de la linealizabilidad, ese orden no tiene que corresponder exactamente con el tiempo real [2].

La **consistencia causal** conserva el orden entre operaciones relacionadas por causa y efecto. Cuando una operación depende de otra, todos los nodos deben observarlas en ese mismo orden. Las operaciones concurrentes sin relación causal pueden ser observadas de manera diferente [2].

La garantía de lectura de los propios cambios, conocida como **read-your-writes**, asegura que un usuario pueda observar sus propias modificaciones posteriores. Esta propiedad evita que, después de actualizar un perfil o agregar un producto al carrito, la misma sesión reciba una versión anterior [2].

La **consistencia eventual** permite que las réplicas presenten diferencias temporales. Si dejan de producirse escrituras, todas las copias terminan convergiendo. Esta garantía puede mejorar la disponibilidad y reducir la coordinación, pero admite lecturas temporalmente desactualizadas [2].

En términos generales, los modelos más fuertes requieren mayor coordinación, mientras que los modelos más débiles permiten reducir la latencia y favorecer la disponibilidad. Las propuestas recientes buscan disminuir el costo de coordinación sin renunciar por completo a garantías fuertes [1], [3].

2.2. Teorema CAP

CAP relaciona tres propiedades: consistencia, disponibilidad y tolerancia a particiones. Gilbert y Lynch demostraron que, cuando una partición impide la comunicación entre nodos, no es posible garantizar simultáneamente consistencia y disponibilidad [4].

Esto no significa que un sistema seleccione permanentemente dos características de tres. La decisión se presenta específicamente durante una partición. El sistema puede rechazar operaciones para impedir divergencias o continuar respondiendo y aceptar estados temporalmente diferentes [4].

En TiendaTech no existe actualmente una partición entre réplicas porque solo hay una instancia PostgreSQL. Si la aplicación pierde el acceso a ese nodo, no dispone de otra copia operativa y deja de estar disponible.

2.3. PACELC

PACELC amplía CAP al considerar que los compromisos también están presentes durante la operación normal. Si existe una partición, debe elegirse entre disponibilidad y consistencia; si no existe una partición, se presenta un compromiso entre latencia y consistencia [5].

En una **replicación síncrona**, la transacción espera que otro nodo reciba o confirme la modificación antes de finalizar. Esta espera reduce el riesgo de que una operación confirmada exista únicamente en el nodo principal, pero añade comunicación y aumenta el tiempo de respuesta [1], [5].

En una **replicación asíncrona**, el nodo principal puede confirmar primero y actualizar después a los seguidores. Esto reduce la latencia de escritura, pero permite que una réplica muestre temporalmente un valor anterior [1], [5].

En TiendaTech, inventario, pedidos y pagos justifican una mayor coordinación porque una inconsistencia podría causar sobreventa, pedidos incompletos o cobros duplicados. Las imágenes, descripciones, recomendaciones y reportes pueden tolerar retrasos breves. Esta distinción corresponde a un análisis propio de los requisitos del dominio.

2.4. Estrategias de replicación

La replicación consiste en mantener copias de los datos en varios nodos para mejorar disponibilidad, tolerancia a fallos y capacidad de lectura. Sin embargo, cada copia adicional introduce la necesidad de propagar, ordenar y confirmar modificaciones [1].

En la replicación **primario-seguidor**, un nodo principal recibe las escrituras y transmite los cambios a uno o varios nodos secundarios. Los seguidores pueden atender consultas y uno de ellos podría sustituir al primario si la arquitectura incorpora mecanismos adicionales de detección de fallos, promoción y redirección de conexiones [1].

La replicación **multi-líder** permite que varios nodos acepten escrituras. Puede reducir la latencia para usuarios ubicados en regiones diferentes, pero genera el riesgo de modificaciones concurrentes sobre los mismos datos. GeoGauss utiliza una arquitectura multi-maestro que combina replicación y control de concurrencia para conservar consistencia fuerte con menor coordinación [3].

En la replicación **sin líder**, las operaciones se distribuyen entre diferentes copias mediante quórums de lectura y escritura, sin depender permanentemente de un nodo principal. Esta estrategia favorece la disponibilidad, pero obliga a manejar versiones concurrentes y reconciliación de datos [6].

Dynamo constituye un caso representativo de replicación sin líder. Su diseño distribuye los datos entre varios nodos, emplea cantidades configurables de réplicas para lectura y escritura y permite reconciliar posteriormente versiones concurrentes. El sistema prioriza disponibilidad y tolerancia a fallos, aceptando consistencia eventual [6].

La replicación también puede ser síncrona o asíncrona. En la modalidad síncrona, la confirmación espera la participación de otro nodo. En la modalidad asíncrona, el primario responde antes de que todas las copias se actualicen. La primera ofrece mayor protección de las operaciones confirmadas; la segunda reduce la latencia, pero admite retraso entre réplicas [1], [5].

2.5. Estado del arte

Margara et al. presentan un modelo para estudiar sistemas distribuidos intensivos en datos según aspectos como distribución, procesamiento, consistencia, recuperación y coordinación. Su revisión muestra que no existe una arquitectura universalmente superior, porque cada sistema combina garantías y costos según sus objetivos [1].

GeoGauss propone una base de datos SQL geo-replicada con arquitectura multi-maestro y consistencia fuerte. Su diseño combina replicación, procesamiento por épocas y control optimista de concurrencia con el propósito de reducir la coordinación entre regiones [3].

PolyBase analiza la influencia de la ubicación geográfica de los líderes y de los grupos de consenso sobre la latencia. La propuesta permite reasignar filas entre grupos de replicación para acercarlas a las regiones desde las cuales reciben más solicitudes [7].

Stabilizer permite que la aplicación defina las garantías de consistencia requeridas para determinadas operaciones. Este planteamiento resulta pertinente para TiendaTech porque los datos de inventario y pagos no requieren las mismas garantías que las recomendaciones o los reportes [8].

Dynamo y Spanner representan dos extremos del espectro. Dynamo prioriza disponibilidad mediante replicación sin líder y consistencia eventual [6]. Spanner ofrece consistencia externa para transacciones distribuidas mediante replicación coordinada y sincronización temporal [9].

La comparación entre ambos sistemas demuestra que no existe una estrategia única para todos los dominios. Dynamo acepta divergencias temporales para continuar respondiendo, mientras que Spanner asume un mayor costo de coordinación para proporcionar un orden global más estricto [6], [9].

Para TiendaTech, una arquitectura completamente eventual sería riesgosa para inventario, pedidos y pagos. En el extremo contrario, una infraestructura global comparable con Spanner sería excesiva para el alcance actual del proyecto. La solución debe aplicar garantías diferentes según la criticidad de los datos. Esta conclusión constituye un análisis propio del dominio apoyado en los compromisos descritos por las fuentes anteriores.

3. Análisis del modelo implementado en TiendaTech

3.1. Caracterización de la implementación

La implementación utiliza una única instancia PostgreSQL 17. El Docker Compose define el contenedor `tiendatech_postgres`, expuesto en el puerto 5432, y el contenedor `tiendatech_app`, expuesto en el puerto 8080.

La aplicación depende directamente del servicio PostgreSQL. No existen configuraciones de nodos secundarios, quórum, consenso, replicación ni promoción automática.

El volumen `postgres_data` conserva los archivos de PostgreSQL cuando el contenedor se reinicia o se vuelve a crear. Sin embargo, no constituye una réplica porque no mantiene una segunda instancia operativa.

La caracterización correcta del modelo es:

TiendaTech utiliza una base de datos PostgreSQL centralizada, con consistencia transaccional local y sin una estrategia de replicación implementada.

3.2. Topología de persistencia

La Figura 1 presenta la topología realmente implementada. El usuario accede a la aplicación, esta realiza operaciones mediante JDBC/SQL y PostgreSQL conserva sus archivos en el volumen Docker.

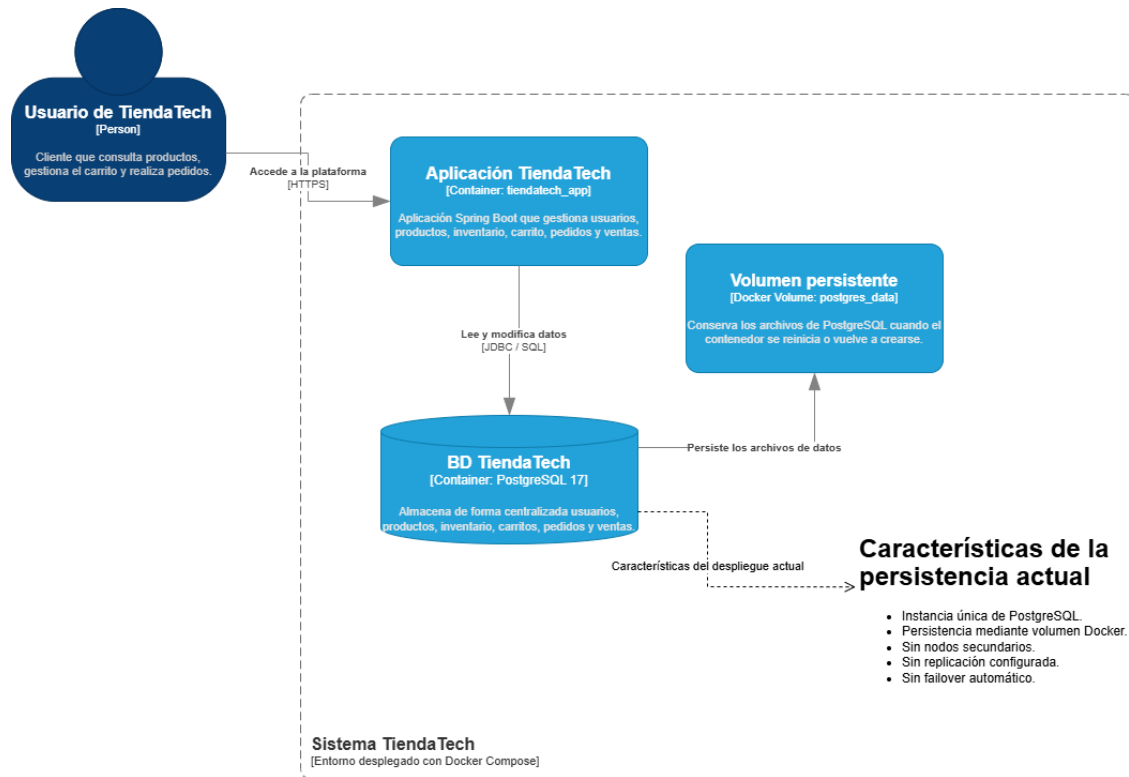


Figura 1: Topología actual de persistencia de TiendaTech. Fuente: elaboración propia.

Como se observa en la Figura 1, PostgreSQL constituye el único nodo de datos. No existen nodos secundarios, replicación ni *failover* automático.

El volumen persistente protege la información frente a la recreación del contenedor, pero continúa dependiendo del mismo servidor. Si se pierde la máquina, el almacenamiento o la instancia PostgreSQL, la aplicación no puede utilizar otra copia.

3.3. Modelo de consistencia identificado

TiendaTech obtiene consistencia transaccional local porque todas las operaciones se ejecutan sobre una misma instancia PostgreSQL. No existe retraso de propagación entre réplicas.

No sería correcto afirmar que el sistema implementa linealizabilidad distribuida, debido a que no existen varios nodos coordinados. Tampoco puede asegurarse que todas las transacciones sean serializables sin revisar el nivel de aislamiento configurado.

La ventaja del nodo único es que no aparecen conflictos entre copias. La limitación es que la consistencia se obtiene sin redundancia y toda la aplicación depende de la disponibilidad de PostgreSQL.

3.4. Estrategia de replicación actual

La estrategia de replicación implementada es **inexistente**. Esta conclusión se basa en el Docker Compose, donde solamente se declara un contenedor PostgreSQL:

Listing 1: Fragmento representativo del Docker Compose de TiendaTech

```

1 services:
2   tiendatech_app:
3     build: .
4     ports:
5       - "8080:8080"
6     depends_on:
7       - tiendatech_postgres
8     restart: always
9
10  tiendatech_postgres:
11    image: postgres:17
12    ports:
13      - "5432:5432"
14    volumes:
15      - postgres_data:/var/lib/postgresql/data
16    restart: always
17
18 volumes:
19   postgres_data:

```

La ausencia de replicación reduce la complejidad, el consumo de recursos y el esfuerzo de administración. Esta decisión resulta comprensible durante una etapa académica inicial.

Sin embargo, PostgreSQL se convierte en un punto único de fallo. La opción `restart: always` intenta reiniciar el mismo contenedor, pero no reemplaza el nodo si falla el servidor o el almacenamiento.

3.5. Requisitos de consistencia del dominio

El inventario requiere garantías estrictas. Si dos usuarios intentan comprar la última unidad y la comprobación del *stock* no se coordina correctamente, el sistema podría aceptar una sobreventa. Por ello, la lectura, comprobación y disminución del *stock* deben formar parte de una operación transaccional coherente.

Los sistemas OLTP geo-replicados muestran que la modificación concurrente de datos compartidos requiere mecanismos de coordinación y control de concurrencia [3].

Los pedidos y las ventas necesitan atomicidad para evitar registros parciales. La cabecera, los detalles, el total y la actualización de existencias deben mantenerse coherentes. Esta necesidad se deriva directamente de las reglas de negocio de TiendaTech.

Los pagos requieren idempotencia y almacenamiento consistente para impedir cobros duplicados. Los usuarios, roles y credenciales también necesitan que las modificaciones relevantes sean visibles rápidamente.

El catálogo, las imágenes y los reportes pueden tolerar retrasos breves. Mostrar temporalmente una descripción anterior tiene menor impacto que confirmar una compra sin *stock*.

La literatura reciente sobre consistencia definida por la aplicación respalda la posibilidad de emplear garantías diferentes según el tipo de operación [8].

Cuadro 1: Nivel de consistencia requerido por tipo de dato en TiendaTech

Nivel requerido	Datos principales
Consistencia estricta	Inventario, pedidos, pagos, ventas, credenciales y roles
Tolerancia limitada	Productos, categorías, marcas, imágenes y búsquedas
Consistencia eventual	Estadísticas, recomendaciones y reportes analíticos

3.6. Análisis mediante CAP

En la topología actual no existe una partición entre réplicas porque solamente se utiliza una instancia PostgreSQL. Si la aplicación pierde acceso al nodo, el servicio de datos deja de estar disponible.

El diseño evita divergencias entre copias, pero no ofrece tolerancia frente al fallo del nodo. Si se incorporaran réplicas, inventario y pagos deberían priorizar consistencia durante una partición, mientras que catálogo y reportes podrían mantener disponibilidad mediante lecturas temporalmente atrasadas. Esta decisión se relaciona con el compromiso formal planteado por CAP [4].

3.7. Análisis mediante PACELC

Durante el funcionamiento normal, TiendaTech no espera confirmaciones remotas, lo que reduce la latencia.

Una réplica síncrona aumentaría el tiempo de confirmación porque la escritura debería esperar la participación de otro nodo. Una réplica asíncrona reduciría ese tiempo, pero podría entregar valores anteriores. Este compromiso corresponde al equilibrio entre latencia y consistencia descrito por PACELC [5].

La ubicación de los líderes y de los grupos de consenso también puede influir en el costo de comunicación y en la latencia de las operaciones [7].

La decisión más adecuada consiste en aplicar garantías fuertes a los procesos críticos y permitir menor coordinación en consultas que toleren retrasos. Esta postura se fundamenta en los requisitos específicos de TiendaTech.

4. Evaluación de alternativas

4.1. Primario con respaldo síncrono

Esta alternativa mantiene un PostgreSQL primario y agrega un nodo de respaldo. Las transacciones críticas se confirman después de que el secundario recibe la modificación.

La replicación síncrona reduce el riesgo de que una operación confirmada exista únicamente en el nodo principal, aunque añade comunicación y tiempo de espera [1], [5].

Esta estrategia resulta apropiada para inventario, pedidos y pagos. Su desventaja es el incremento de latencia y la necesidad de implementar detección de fallos, promoción del secundario y redirección de conexiones.

4.2. Primario con réplica asíncrona

Esta alternativa conserva un primario para las escrituras y añade uno o varios seguidores para las lecturas.

Las réplicas podrían atender consultas de catálogo, imágenes, búsquedas y reportes. Sin embargo, pueden mostrar información temporalmente atrasada, por lo que no deberían utilizarse para confirmar el *stock* definitivo de una compra.

La ventaja principal es que se distribuye la carga de lectura sin obligar al primario a esperar la confirmación de todos los seguidores [1].

4.3. Estrategia híbrida

La estrategia híbrida combinaría:

- un nodo primario para todas las escrituras;
- un respaldo síncrono para operaciones críticas;

- una réplica asíncrona para catálogo y reportes;
- un mecanismo de *failover*;
- monitoreo del retraso de replicación;
- copias de seguridad externas.

Esta alternativa diferencia las garantías según la criticidad de los datos, principio relacionado con la consistencia definida por la aplicación [8].

Ofrece un mejor equilibrio entre consistencia, disponibilidad y rendimiento, aunque incrementa la complejidad de administración y enrutamiento de consultas.

4.4. Tabla comparativa

Cuadro 2: Comparación entre el modelo actual y las alternativas evaluadas

Criterio	Nodo único	Respaldo síncrono	Réplica asíncrona	Estrategia híbrida
Consistencia	Local	Alta	Alta en el primario	Alta en datos críticos
Disponibilidad	Baja	Alta con failover	Alta para lecturas	Alta
Latencia de escritura	Baja	Mayor	Baja	Media
Latencia de lectura	Limitada	Media	Baja	Ajustada al dato
Tolerancia a fallos	Baja	Alta	Media	Alta
Tolerancia a particiones	No implementada	Prioriza consistencia	Mantiene lecturas	Depende del proceso
Riesgo de datos atrasados	No aplica	Muy bajo	Presente	Controlado
Escalabilidad de lectura	Limitada	Media	Alta	Alta
Complejidad operativa	Baja	Media	Media	Alta
Adecuación general	Prototipo	Datos críticos	Consultas no críticas	Producción

5. Discusión crítica

5.1. ¿Es óptimo el modelo actual?

El modelo actual es adecuado para una etapa académica porque simplifica el desarrollo y evita problemas de sincronización entre réplicas.

Sin embargo, no es óptimo para una plataforma productiva. PostgreSQL constituye un punto único de fallo y todas las operaciones dependen de su disponibilidad.

El volumen Docker conserva los archivos de la base, pero no proporciona una segunda instancia operativa. De igual manera, `restart: always` intenta reiniciar el mismo contenedor y no ejecuta *failover* hacia otro servidor.

Por tanto:

El modelo centralizado es suficiente para el prototipo actual, pero no es óptimo para una versión productiva que requiera alta disponibilidad, escalabilidad y tolerancia a fallos.

5.2. Trade-offs

El primer compromiso es simplicidad frente a disponibilidad. Una instancia única es sencilla de administrar, pero su falla detiene toda la persistencia.

El segundo compromiso es latencia frente a durabilidad. Una confirmación síncrona protege mejor las transacciones, pero añade comunicación y tiempo de espera [5].

El tercero es disponibilidad frente a actualidad de las lecturas. Una réplica asíncrona puede distribuir consultas, pero podría entregar valores temporalmente anteriores [1].

El cuarto es consistencia uniforme frente a consistencia diferenciada. Aplicar coordinación fuerte a todos los datos sería innecesario, mientras que aplicar consistencia eventual al inventario y a los pagos sería riesgoso. La consistencia definida según la aplicación permite adaptar las garantías a cada tipo de dato [8].

5.3. Limitaciones

Las principales limitaciones identificadas son:

- Punto único de fallo.
- Ausencia de *failover* automático.
- Escalabilidad horizontal limitada.
- Dependencia del mismo servidor y volumen.
- Posibles interrupciones durante mantenimiento.
- Falta de redundancia operativa.
- Imposibilidad de distribuir las lecturas.

5.4. Mitigaciones

Se proponen las siguientes medidas:

1. Mantener transacciones para controlar la comprobación y reducción del *stock*.
2. Incorporar inicialmente una réplica asíncrona.
3. Evaluar un respaldo síncrono para pagos y pedidos.
4. Mantener copias de seguridad externas.
5. Monitorear la disponibilidad y el retraso de replicación.
6. Implementar idempotencia en pagos y creación de pedidos.
7. Probar periódicamente los procedimientos de *failover* y restauración.

5.5. Propuesta recomendada

La propuesta recomendada es una evolución híbrida y progresiva. Primero se añadiría una réplica asíncrona para catálogo, reportes y recuperación. Posteriormente, se incorporaría un respaldo síncrono para proteger inventario, pedidos y pagos.

Las consultas críticas permanecerían en el primario, mientras que las consultas tolerantes a retrasos podrían dirigirse a las réplicas.

Esta alternativa reduce progresivamente el punto único de fallo sin introducir desde el inicio la complejidad de una arquitectura multi-líder.

6. Conclusiones

TiendaTech utiliza una única instancia PostgreSQL y no posee replicación. Su modelo corresponde a una persistencia centralizada con consistencia transaccional local.

La solución es adecuada para un prototipo académico, pero presenta un punto único de fallo y no garantiza continuidad operativa.

El análisis mediante CAP muestra que, al perder el acceso a PostgreSQL, la aplicación deja de estar disponible. PACELC permite observar que la baja latencia actual se obtiene al no coordinar varias réplicas.

La replicación síncrona resulta apropiada para inventario, pedidos y pagos, aunque incrementa la latencia. La replicación asíncrona puede mejorar la capacidad de lectura del catálogo y de los reportes, pero admite información temporalmente atrasada.

El modelo actual no es óptimo para una versión productiva. La alternativa recomendada es una estrategia híbrida que combine garantías fuertes para los procesos críticos y réplicas asíncronas para las lecturas no críticas.

7. Declaración de uso de inteligencia artificial generativa

Para la elaboración de este informe se utilizó ChatGPT como herramienta de apoyo en la organización de la estructura del documento, la revisión de la claridad de la redacción y la presentación de algunas ideas técnicas. La asistencia se aplicó principalmente en las secciones de marco teórico, evaluación de alternativas, discusión crítica y conclusiones.

El análisis de la arquitectura de TiendaTech, la comprobación de la implementación, la interpretación de las fuentes, la selección de la propuesta y la revisión final fueron realizadas por el autor.

Referencias

- [1] A. Margara, G. Cugola, N. Felicioni y S. Cilloni, “A Model and Survey of Distributed Data-Intensive Systems,” *ACM Computing Surveys*, vol. 56, n.º 1, págs. 1-69, 2023. DOI: [10.1145/3604801](https://doi.org/10.1145/3604801).
- [2] P. Viotti y M. Vukolić, “Consistency in Non-Transactional Distributed Storage Systems,” *ACM Computing Surveys*, vol. 49, n.º 1, págs. 1-34, 2016. DOI: [10.1145/2926965](https://doi.org/10.1145/2926965).
- [3] W. Zhou et al., “GeoGauss: Strongly Consistent and Light-Coordinated OLTP for Geo-Replicated SQL Database,” *Proceedings of the ACM on Management of Data*, vol. 1, n.º 1, págs. 1-27, 2023. DOI: [10.1145/3588916](https://doi.org/10.1145/3588916).
- [4] S. Gilbert y N. Lynch, “Brewer’s Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services,” *ACM SIGACT News*, vol. 33, n.º 2, págs. 51-59, 2002. DOI: [10.1145/564585.564601](https://doi.org/10.1145/564585.564601).
- [5] D. J. Abadi, “Consistency Tradeoffs in Modern Distributed Database System Design: CAP is Only Part of the Story,” *Computer*, vol. 45, n.º 2, págs. 37-42, 2012. DOI: [10.1109/MC.2012.33](https://doi.org/10.1109/MC.2012.33).
- [6] G. DeCandia et al., “Dynamo: Amazon’s Highly Available Key-value Store,” en *Proceedings of the 21st ACM SIGOPS Symposium on Operating Systems Principles*, 2007, págs. 205-220. DOI: [10.1145/1294261.1294281](https://doi.org/10.1145/1294261.1294281).
- [7] C. Ruan et al., “PolyBase: Adapting to Data Affinity Changes in Geo-Replicated Database via Row-Level Consensus-Group Affiliation Re-Assignment,” *Proceedings of the VLDB Endowment*, vol. 18, n.º 3, págs. 702-714, 2024. DOI: [10.14778/3712221.3712236](https://doi.org/10.14778/3712221.3712236).

- [8] P. Li, L. Pan, X. Yang, W. Song, Z. Xiao y K. Birman, “Stabilizer: Geo-Replication with User-Defined Consistency,” en *2022 IEEE 42nd International Conference on Distributed Computing Systems*, 2022, págs. 359-369. DOI: [10.1109/ICDCS54860.2022.00042](https://doi.org/10.1109/ICDCS54860.2022.00042).
- [9] J. C. Corbett et al., “Spanner: Google’s Globally Distributed Database,” *ACM Transactions on Computer Systems*, vol. 31, n.º 3, págs. 1-22, 2013. DOI: [10.1145/2491245](https://doi.org/10.1145/2491245).